

# Log File Analyzer

## Project Report

**Name:** Ishaan Gupta

**Roll No:** 24B1081

# 1. Introduction

This project titled **Log File Analyzer** is a Flask-based web application designed to automate the process of parsing and analyzing Apache-style log files. Users can upload '.log' files, filter data by timestamp, log level, or event code, and visualize the results using interactive plots.

The application is especially useful for system administrators, developers, and cybersecurity analysts who want a lightweight but powerful tool to visualize trends, detect anomalies, or understand user behavior from server logs.

In many production environments, server logs grow at a rapid pace and contain critical insights related to system performance, security incidents, and software errors. However, manually sifting through hundreds or thousands of log entries is tedious, error-prone, and time-consuming. The need for an efficient, user-friendly system to automate log parsing, filtering, and visualization inspired the development of this project.

Traditional log analysis tools are often expensive, complex to set up, or tailored toward specific use cases. In contrast, this web application focuses on simplicity, portability, and extensibility. It uses minimal external dependencies and provides a clean, intuitive web interface where users can easily manage and analyze their server logs without deep technical expertise.

A notable design choice in the project is the use of Bash scripts for log preprocessing. This allows parsing and filtering to happen efficiently at the command-line level, making the system lightweight and fast. Additionally, it ensures that the application can run even on servers with limited Python packages installed, leveraging native shell capabilities instead.

Visualization is another key focus area of the project. By providing graphical representations like line plots, pie charts, and bar graphs, users can quickly identify patterns and anomalies that would be difficult to spot in raw text form.

Overall, the **Log File Analyzer** project aims to bridge the gap between basic manual log inspection and complex enterprise-grade log management solutions. It is ideal for small to medium-sized deployments, educational purposes, or as a foundation to build more advanced analytics systems tailored to specific organizational requirements.

## 2. Running Instructions

Follow the steps below to run the project locally:

1. Ensure you have Python, Flask, Bash, and Matplotlib installed.
2. Ensure you must run this on Ubuntu terminal with gawk installed.
3. Navigate to the project root and run the command:

```
python app.py
```

4. Open your browser and go to `http://localhost:5000`
5. Use the interface to upload '.log' files, apply filters, and view the results.
6. You can also download the processed or filtered CSV files and plots.

### 3. Website Layout

We operate in 4 webpages:

- upload.html
- landing.html
- plots.html
- display.html

The website consists of several key pages accessible through defined Flask routes:

- **/upload** – Upload ‘.log’ files for processing.
- **/display/{filename}** – Display the content of the processed log in tabular format.
- **/plots/{filename}** – Generate predefined line, pie, and bar plots.

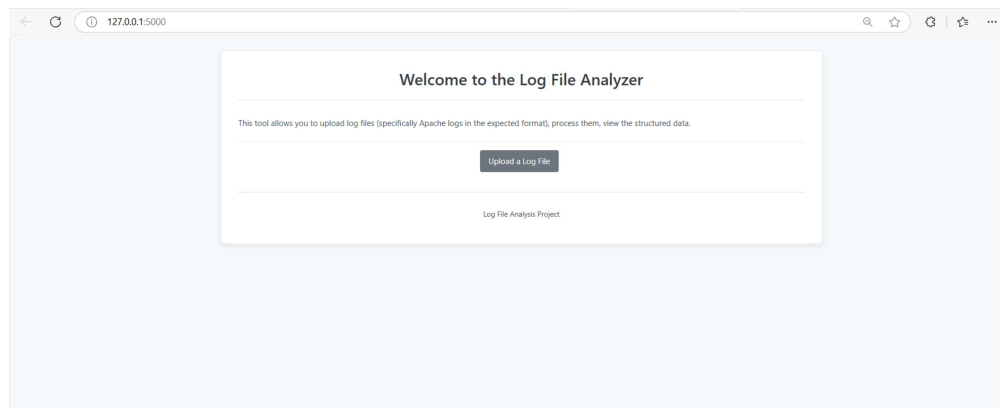


Figure 1: landing page screenshot

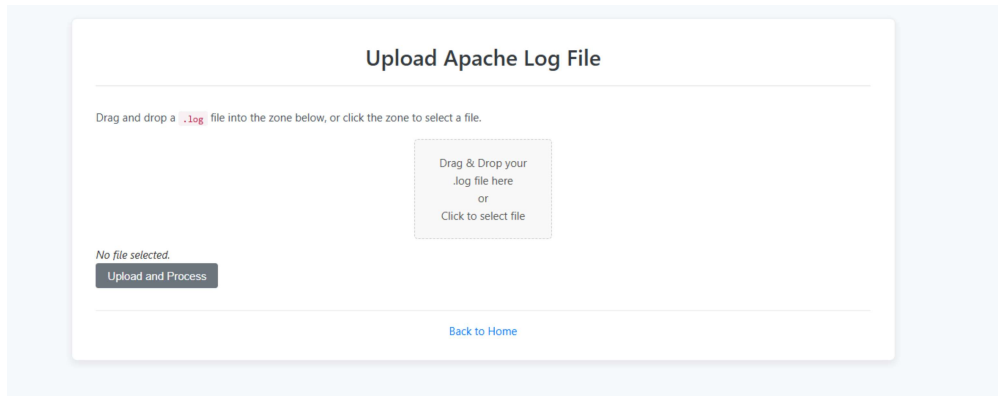


Figure 2: Upload page screenshot

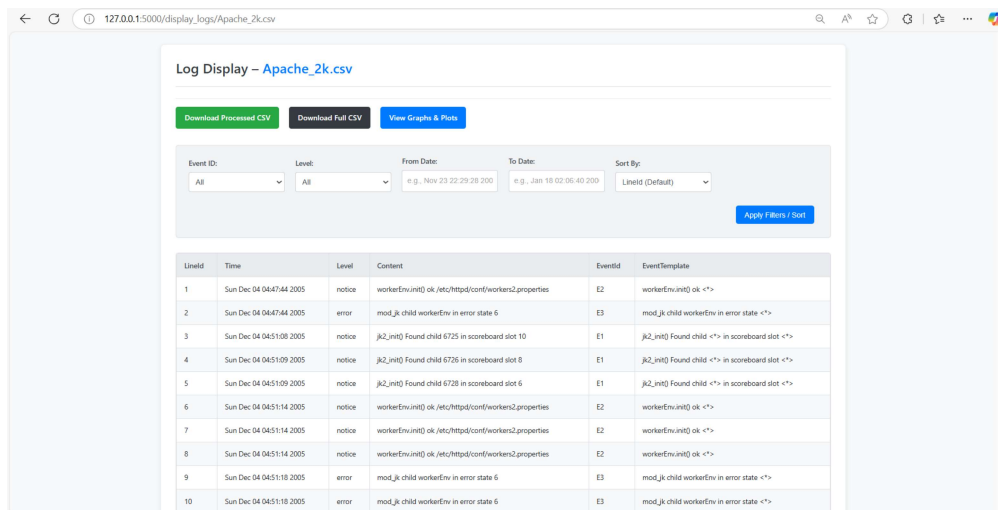


Figure 3: Display page screenshot

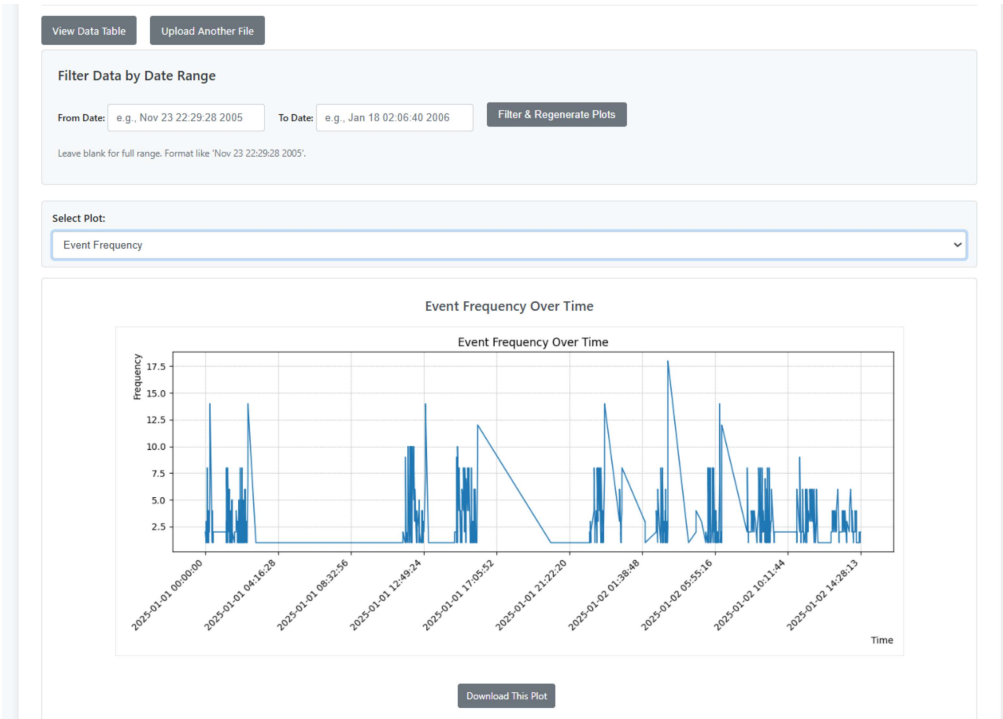


Figure 4: line plot

## 4. Modules

The project makes use of a combination of Python libraries and Bash utilities to achieve seamless log file analysis. This hybrid approach ensures that the system remains lightweight while being powerful enough for flexible operations.

The following Python modules were primarily used to build and extend the core functionality of the application:

- **flask** – The Flask web framework is responsible for creating routes, handling user requests, rendering templates, and managing sessions. It acts as the backbone of the server-side logic, making the application accessible through a simple and intuitive web interface.
- **os, subprocess** – The **os** module is used for interacting with the file system, such as creating directories and handling file paths. The **subprocess** module plays a crucial role in executing Bash scripts for log processing and filtering directly from the Python backend.
- **matplotlib.pyplot** – This module is used for generating visualizations including line graphs, pie charts, and bar charts. The graphs help users better understand log trends and distributions over time, levels, or event types. The application uses Matplotlib in non-interactive (headless) mode to automatically save plots as image files.
- **matplotlib.ticker** – It provides tools to control the placement and formatting of ticks on plot axes. In `app.py`, `MaxNLocator` from `matplotlib.ticker` is used to ensure that axis ticks (especially on the y-axis or time-axis) appear at integer values only, improving readability. This is particularly useful when plotting counts or frequencies, where fractional ticks don't make sense. By using `MaxNLocator(integer=True)`, the plot avoids non-integer tick labels and maintains a clean, logical axis.
- **numpy** - it increases efficiency by directly helping in manipulating data structures without having to use for loops in flask script while plotting with respect to time the event frequency.

### Bash and Shell Modules:

In addition to Python modules, the project heavily relies on Bash scripting and standard Unix utilities for high-performance file manipulation:

- **date** – Used for parsing and reformatting timestamps found in Apache log entries to ensure consistency and compatibility in the CSV output.
- **awk** – A simple yet powerful field processing tool used in scripts to parse fields, check logs and split strings is heavily used throughout the bash scripts and forms a solid bash awk pipeline.
- **sed** – Used for stream editing, such as replacing, modifying, or extracting parts of log entries while processing them into a structured format.
- **tr** – A lightweight utility used to translate or delete characters, mainly for cleaning and standardizing log fields before conversion to CSV.
- **regex** – Regular expressions are heavily used within Bash scripts to accurately parse the Apache log structure and extract relevant fields like timestamp, level, and message, even when log messages include special characters or whitespace.

*Note:* Popular modules like **pandas**, **csv**, **collections** were avoided. This decision ensures that the system remains lightweight, portable across systems with minimal dependencies, and capable of functioning even on resource-constrained environments. The use of native Bash tools for parsing ensures faster performance compared to heavier Python-based processing for this scale of log files. The sorting and filtering happens in the backend from bash script (filter.sh) after accepting the form input by "POST". This reduces dependency on local server to handle complex operations. Although user experience is little compromised, it is useful when we have log files having large number of lines.



## 5. Directory

## Structure

The project is structured as follows:

- **app.py** – Main Flask server script.
- **bash\_scripts**
  - **check.sh** – Checks the validity of log file and if found, directs it to process.sh
  - **process.sh** – Converts Apache-style log files to structured CSV format.
  - **check.awk** – Checks the format by evaluating regex matching of each line (called in check.sh)
  - **parse.awk** – Slices the fields and prints the structured CSV format (called in process.sh)
- **templates/** – HTML templates for all views.
  - **upload.html**, **display.html**, **plots.html**, etc.
- **static/**
  - **css/styles.css** – Custom styling for web views.
  - **images/** – Contains generated plot images (line, pie, bar, custom).
- **uploads/** – Stores all ‘.log’ files uploaded
- **processed/** – Contains:
  - Converted CSV files (e.g., **Apache\_2k.csv**).
  - Filtered CSVs prefixed with **filtered\_**.

## 6. Basic & Advanced Features

### 6.1. Basic Features

- Parse, check and convert logs into CSV without using Python parsing modules.
- Display log entries in an interactive HTML table.
- Display and download the filtered plots generated on the CSV.

### 6.2. Advanced Features

- Filtering based on:
  - Log Level (e.g., error, notice)
  - Event ID (e.g., E1, E2, ...)
  - Timestamp range
- Visualization:
  - **Line Chart** – Event frequency over time.
  - **Pie Chart** – Level distribution.
  - **Bar Chart** – Frequency of different Event IDs.

*Note:* I tried to not use any external library other than matplotlib and numpy for making the table and the input array for graphs

## 7. Project

## Journey

1. First, had problems in setting up a robust pipeline that checks and parses log file to form csv's using bash and awk
2. Then made the app.py script that decided the general workflow of all the templates and scripts at the server level
3. Then fixed issues in plotting graphs using numpy and automatic locator
4. Wrote the javascript and html necessary for upload and display page
5. Implemented visual feedback and error messages for better UX.

### Challenges:

- Handling reading lists of dictionaries for making the table using simple python
- Creating secure and safe shell execution from Flask.
- Ensuring platform-independence in shell scripts

### Learnings:

- The use of flask as a medium to communicate with the server and connect all pages.
- Writing portable Bash scripts and integrating them with Python.
- Using matplotlib in non-interactive (server) mode for image generation.

## 8. Bibliography

- Flask Documentation – <https://flask.palletsprojects.com/>
- Matplotlib – <https://matplotlib.org/stable/contents.html>
- Apache Log Reference – <https://httpd.apache.org/docs/current/logs.html>
- Bash Guide – <https://tldp.org/LDP/abs/html/>
- Numpy – <https://numpy.org/doc/>